Praktikum
Digitale Übertragungstechnik

SoSe 2025

Prof. Matthias Kronauge

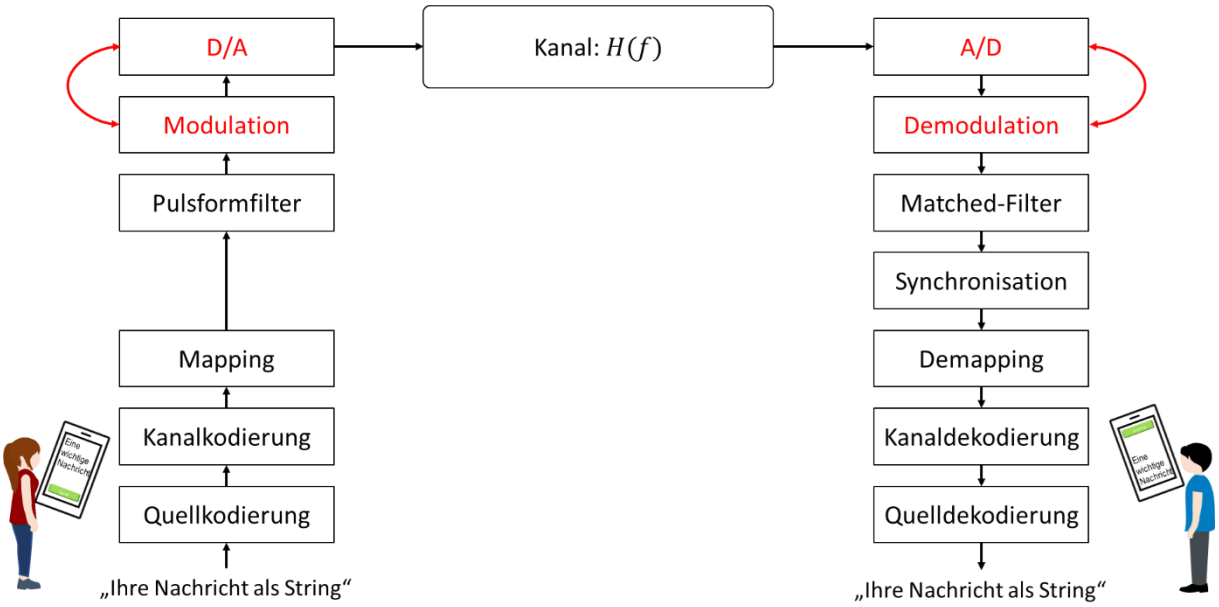
Aufbau eines digitalen Kommunikationssystems

In diesem Praktikum zur Vorlesung Digitale Übertragungstechnik soll nach und nach ein digitales Übertragungssystem realisiert werden.

Die Methoden, die Sie in der Vorlesung dazu kennenlernen basieren auf denen, die Sie bereits aus den Modulen Signale und Systeme 1 und 2 kennen. Zur Implementierung verwenden wir MATLAB. Das hat zwei Vorteile:

- Es ist sehr einfach die verwendeten Signale im Zeit- und Frequenzbereich zu Plotten und zu analysieren
- Die Implementierung ist einfacher als z.B. in C, da viele Funktionen wie die Faltung oder die FFT direkt verfügbar sind und die Handhabung von Arrays und Datentypen vereinfacht wird.

Außerdem soll die verwendete Hardware möglichst einfach gehalten werden. Daher verwenden wir als Digital-Analog-Wandler im Sender und Analog-Digital-Wandler im Empfänger jeweils eine USB-Soundkarte mit einer Abtastrate von $f_{sa} = 48\,kHz$. Um darüber hinaus keine weitere Hardware in Form z.B. eines Modulators zu benötigen, vertauschen wir die Reihenfolge von D/A-Wandler und Modulator im Sender und A/D-Wandler und Demodulator im Empfänger und führen die Modulation und Demodulation digital durch – wie im folgenden Bild dargestellt.



Als Kanal dient zunächst ein Audiokabel, das später mit Lautsprecher und Mikrophon durch die Luft ersetzt werden kann.

Legen Sie sich für dieses Praktikum einen Ordner namens „*CommunicationSystem*“ auf dem PC an. Legen Sie außerdem einen Unterordner namens „*functions*“ an, in dem Sie dann alle Funktionen als MATLAB Dateien abspeichern können.

In Ihrem Hauptordner „*CommunicationSystem*“ erstellen Sie nun eine Datei namens „*SingleCarrierCommunication.m*“, in der Sie das Sende- und Empfangssystem implementieren können.

Schreiben Sie als erstes einen Header in dieser Datei, der Informationen über den Inhalt, Ihre Namen etc. enthält.

Es empfiehlt sich ganz am Anfang die Kommandozeile zu löschen, den Workspace (alle angelegten Variablen) zu löschen und alle geöffneten Fenster zu schließen.

Schließlich müssen Sie noch Ihren Unterordner einbinden um später auf Funktionen, die Sie darin speichern verwenden zu können.

Dann kann es losgehen und Sie definieren eine Nachricht, die Sie versenden wollen als Text-String.

Die ersten Zeilen Ihrer Hauptdatei können dann so aussehen:

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%
% Single Carrier Communication
% Name: …
% Datum: …
%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
clc
clear
close all
addpath('functions\');

%% Definieren Sie eine Nachricht,
% die versendet werden soll:
msg = 'Hallo hier ist eine wichtige Nachricht!';

% Diese Nachricht stellt die Quelle für unsere Kommunikation dar.
```

Aufgabe 1 - Quellkodierung

Zunächst muss über die Quellkodierung der Text in einen Bitstrom übersetzt werden. Um es einfach zu halten, wird ein Code fester Länge verwendet. Groß- und Kleinbuchstaben, Ziffern sowie Satzzeichen und die meisten Sonderzeichen befinden sich im ASCII-Code im Bereich 0…127 – können also mit nur 7 Bit einfach abgebildet werden. Bei dieser 7 Bit Kodierung verzichten wir auf Umlaute und auf das ‚ß'. Damit bleibt ein Bit in jedem Byte übrig, das später zur Kanalkodierung genutzt werden kann.

Legen Sie in Ihrem Unterordner ‚*functions*' folgende Dateien an:

- ‚*sourceCoding.m*'
- ‚*sourceDecoding.m*'
- ‚*decToBinVec.m*'
- ‚*binVecToDec.m*'

In MATLAB kommen Sie sehr leicht von einem character z.B. ‚a' zum entsprechenden ASCII-Code als Dezimalwert. Eine Binärzahl gib MATLAB mit der Funktion ‚dec2bin' allerdings nur als String zurück. Wir benötigen aber einen Vektor mit ‚1' und ‚0' als Zahlen, um damit rechnen zu können. Daher die unteren beiden Hilfsfunktionen.

Die erste Hilfsfunktion könnte folgendermaßen aussehen:

```
%% %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%  Name: **** ****
%  Date: **.**.****
%
%  decToBinVec: converts decimal number to binary vector
%
%  Input:   - dec:  decimal number (in the range 0...2^N-1)
%           - N:    length of the output vector (number of bits)
%  Output:  - bin:  vector of bit values (0 or 1) MSB at index 1
%  Example: decToBinVec(12,7) yields: [0 0 0 1 1 0 0]
%
%% %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
function bin = decToBinVec(dec,N)
bin = zeros(1,N);    % prepare output vector with zeros
% for loop to calculate each bit
for i = 1:N
    % calculate each single bit
    bin(i) = bitand(dec,2^(N-i))>0;
end
```

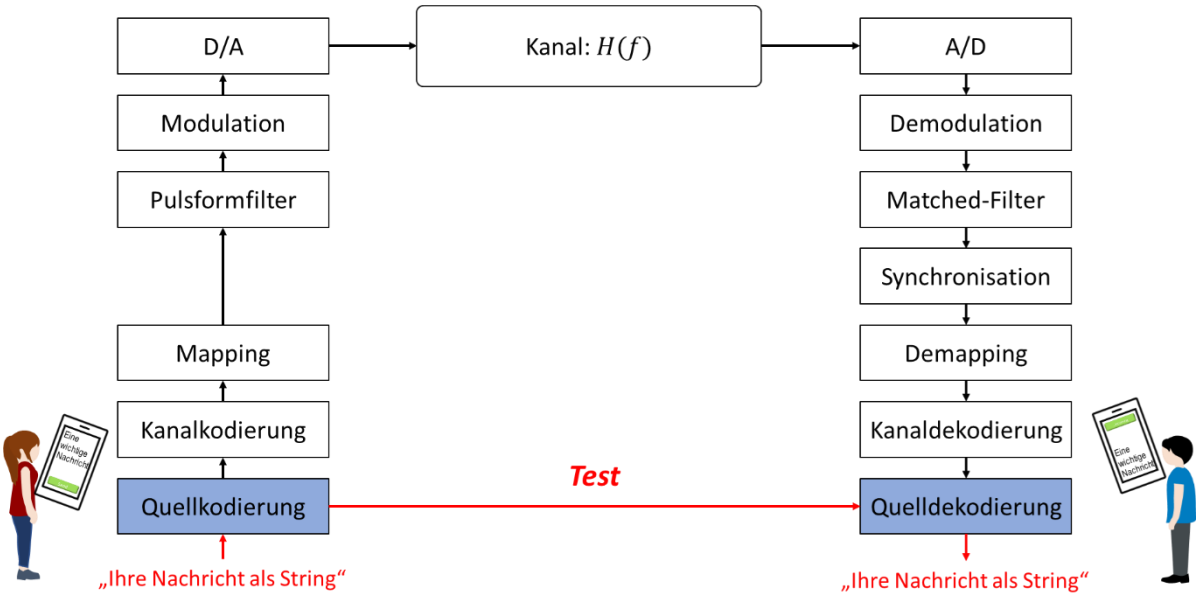
Beachten Sie wie Funktionen in MATLAB in Dateien definiert werden:

```
function [output1, output2] = functionName(input1, input2)
```

Dabei muss der Funktionsname exakt dem Dateinamen entsprechen.

Programmieren Sie so die Inhalte der vier Funktionen. Die Funktion „*sourceCoding*" bekommt den gesamten String der zu sendenden Nachricht und gibt die entsprechenden Bits als Vektor zurück. Diesen Vektor können Sie dann (wie in folgendem Bild dargestellt) zum Testen in die Funktion „*sourceDecoding*" eingeben und müssen Ihre Nachricht wieder als String zurück erhalten.

Diesen Test schreiben Sie in Ihre Hauptfunktion „*SingleCarrierCommunication.m*".


D/A
Kanal: $H(f)$
A/D
Modulation
Demodulation
Pulsformfilter
Matched-Filter
Synchronisation
Mapping
Demapping
Kanalkodierung
Kanaldekodierung
Test
Quellkodierung
Quelldekodierung
„Ihre Nachricht als String“
„Ihre Nachricht als String“

Aufgabe 2 - Kanalkodierung

Als Kanalkodierung soll eine einfache Parity-Matrix – wie aus der Vorlesung bekannt – verwendet werden.

Legen Sie dazu folgende Dateien und Funktionen in Ihrem Unterordner „*functions*“ an:

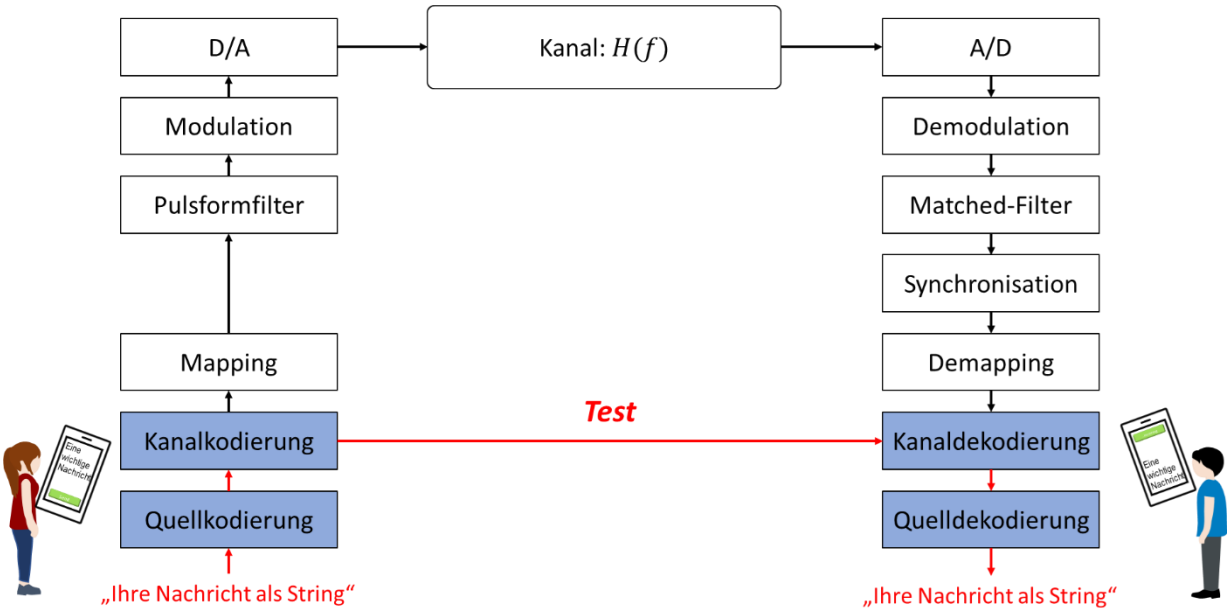
- „*channelCoding.m*“
- „*channelDecoding.m*“

In die Funktion *channelCoding* wird der Bit-Vektor mit der gesamten Quell-kodierten Nachricht eingegeben. Fassen Sie dann in der Funktion immer 7x7 Bit zu einer Matrix zusammen, sodass inklusive Parity-Bits eine 8x8 Bits große Matrix entsteht. 64 Bit lassen sich später gut auf Symbole mappen.

Wenn die Länge der eingegangenen Nachricht nicht einem Vielfachen von 7 entspricht, füllen Sie sie der Einfachheit halber schon nach der Definition des Texts in der Hauptdatei mit einem beliebigen Zeichen wie z.B. ‚*‘ in entsprechender Anzahl auf.

Geben Sie als Rückgabewert bei der Dekodierung neben den dekodierten Bits auch die Anzahl der detektierten Fehler aus und – wenn nur ein Fehler aufgetreten ist – ob ein Fehler korrigiert wurde.

Fügen Sie dem Test der Quellkodierung in Ihrer Hauptfunktion „*SingleCarrierCommunication.m*“ nun auch die Kanalkodierung hinzu – wie im Bild dargestellt:



Aufgabe 3 - Mapping

Definieren Sie zunächst ein Alphabet an Sendesymbolen. Im ersten Schritt soll das Alphabet rein reell sein – z.B.:

```
alphabet = [-3; -1; +1; +3;];
```

Legen Sie dazu folgende Dateien und Funktionen in Ihrem Unterordner „*functions*“ an:

- „*symbolMapping.m*“
- „*symbolDemapping.m*“

Als Eingang für die Funktion werden die Kanal-kodierten Bits eingegeben sowie das zuvor definierte Alphabet. Berechnen Sie aus der Mächtigkeit des Alphabets (Anzahl Symbole) als erstes wie viele Bits pro Symbol übertragen werden können. Damit und mit der Anzahl der eingegangenen Bits können Sie einen Output-Vektor der richtigen Länge anlegen.

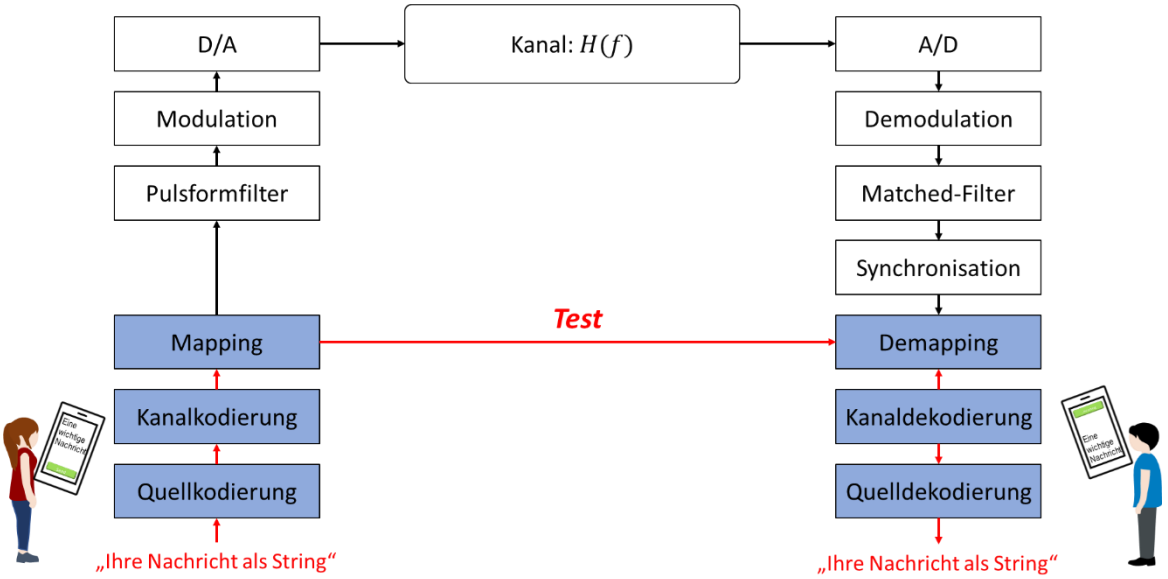
Ebenfalls aus der Mächtigkeit des Alphabets kann direkt die Generatormatrix (und für das Demapping die inverse dazu) erstellt werden. Sie können entweder zu Beginn ein Codebuch (Lookup-Table) erstellen mit allen möglichen Bitkombinationen und den entsprechenden Symbolen oder Sie berechnen die Kodierung jedes Mal direkt mit den Matrizen.

Erweitern Sie das Mapping auch auf komplexe Symbole z.B. für eine 16 QAM. Dazu muss immer je eine Hälfte der Bits für den Realteil und die andere Hälfte für den Imaginärteil Gray-kodiert werden. Ein komplexes (2D) Alphabet können Sie in MATLAB z.B so anlegen:

```
alphabet = [-3 -1 +1 +3] +1i* [-3; -1; +1; +3;];
```

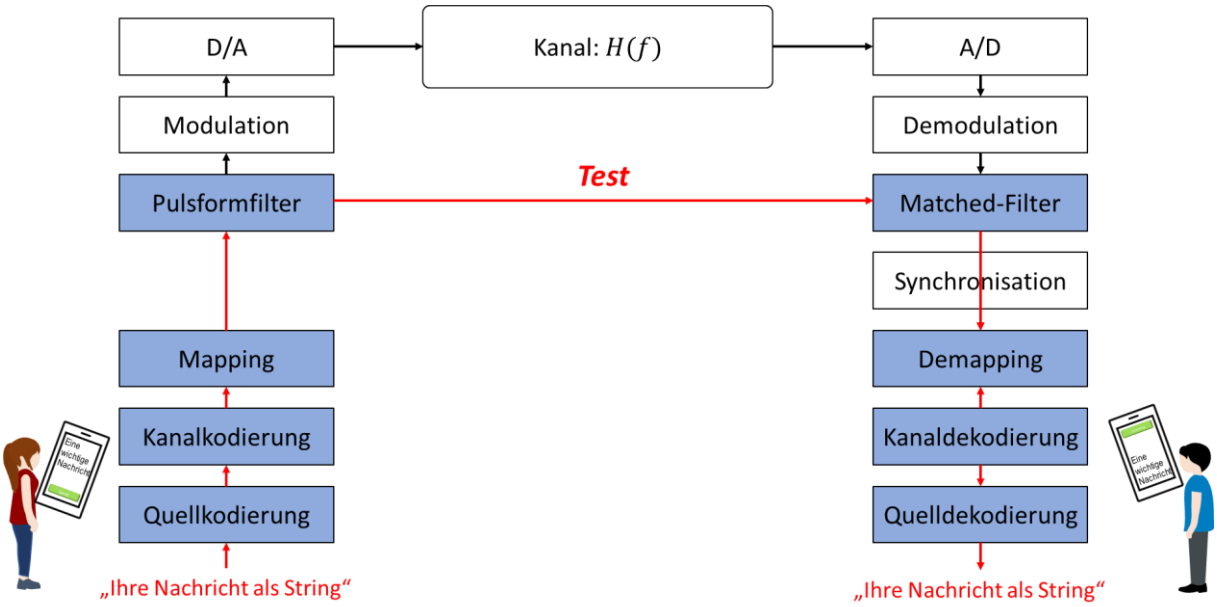
Ausgabewerte der Funktion sind jeweils reelle oder komplexe Symbole, d.h. ein Vektor, der nur Werte aus dem vorgegebenen Alphabet enthält. Beim Demapping entsprechend umgekehrt.

Plotten Sie die komplexen Symbole als Konstellationsdiagramm. (Figure 1)



Aufgabe 4 – Pulsformfilter und Matched Filter

Die zuletzt generierten Symbole sollen nun für die Übertragung über den Kanal vorbereitet werden. Dazu werden die Symbole mit einem Pulsformfilter entsprechend transformiert. Auf dem Kanal kommt unter anderem Rauschen hinzu. Damit die Symbole im Rauschen wieder detektiert werden können, wird das verrauschte Empfangssignal mit der bekannten Sendeimpulsform korreliert. Das Empfangsfilter muss daher dieselbe Impulsantwort wie das Sendepulsformfilter haben. Daher wird es Matched Filter genannt. Zum Testen wird hier der Kreis vom Pulsformfilter auf der Senderseite zum Matched Filter auf der Empfängerseite geschlossen, die Synchronisation zunächst außen vorgelassen, da diese erst wichtig wird, wenn der Kanal ins Spiel kommt:



Erzeugen Sie mit Hilfe der MATLAB-Funktion

‚`h = rcosdesign(alpha, Nsym, Nsam, 'sqrt')`‘

die Filterkoeffizienten eines Root-Raised-Cosine-Filters. Verwenden Sie dafür folgende Werte:

- Länge der Impulsantwort (in Anzahl Symbole) `N_sym = 6`
- Anzahl Abtastwerte pro Symbol `N_sam = 8`
- Roll-off-Faktor `alpha` (kann später variiert werden)

Skalieren Sie die Filterkoeffizienten, so dass die Verstärkung des Filters 1 ist. Dazu teilen Sie alle Filterkoeffizienten durch die Summe der Filterkoeffizienten.

Plotten Sie in eine Figure und zwei Subplots die Impulsantwort des Filters und die Übertragungsfunktion zum Vergleich jeweils für unterschiedliche Werte von `alpha`. (Figure 2)

Filtern Sie Ihre Sendesymbole mit dem Pulsformfilter mit der Funktion

`y = conv(x, h, ‘same‘)`.

Beachten Sie dabei, dass die Abtastrate der Impulsantwort des Filters (der Filterkoeffizienten) und die Symbolrate nicht identisch sind. Mit `N_sam = 8` Abtastwerten pro Symbol ist die Abtastrate der Filterimpulsantwort 8-fach höher als die Symbolrate $f_{sa} = 8 \cdot \frac{1}{T_{sym}}$. Daher müssen zwischen

den Symbolen `N_sam-1` Nullen eingefügt werden. Das geht in MATLAB mit der Funktion `upsample`.

Plotten Sie den Ausgang des Filters und die eingegebenen Symbole. (Figure 3) Achten Sie dabei auf die richtigen Zeitachsen, sodass beide Signale zeitlich korrekt übereinander liegen. Für die Symbole können Sie die Funktion `stem` verwenden.

So wie es aus dem Filter herauskommt kann das Signal grundsätzlich über einen Kanal übertragen werden. Im Empfänger wird als Matched Filter ein Filter mit derselben Impulsantwort verwendet – um im verrauschten Empfangssignal die Sendepulse wiederzufinden. Das Matched Filter (auch Korrelationsfilter oder Optimalfilter) erhöht das Signal-zu-Rausch-Verhältnis, sodass eine Detektion überhaupt möglich wird.

Wenden Sie das Filter ein zweites Mal an und **plotten Sie den Filterausgang. (Figure 3, Subplots)**

Plotten Sie außerdem das Augendiagramm. (Figure 3) Dazu kann die MATLAB-Funktion

```
reshape(y, 3*Nsam,[]);
```

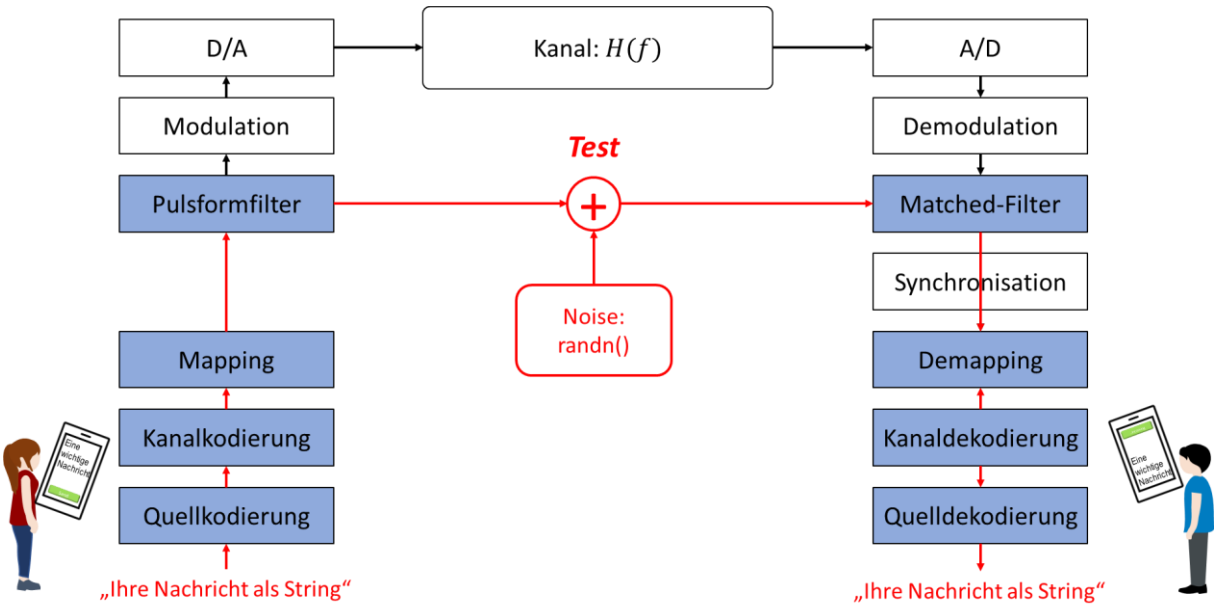
hilfreich sein.

Zum Testen der Übertragungskette nehmen Sie zum richtigen Zeitpunkt im Symboltakt jeweils den Amplitudenwert am Ausgang des Matched-Filters um die Symbole zurückzugewinnen.

Plotten Sie mit diesen Amplituden das Konstellationsdiagramm erneut. (Figure 4)

Runden Sie die Amplitudenwerte auf die Ganzzahligen Symbolwerte aus Ihrem Alphabet und übergeben Sie diese an Ihre Demapping Funktion und in der Folge an die Kanal- und Quelldekodierung. Wenn alles richtig ist, sollten Sie Ihre eingegebene Nachricht zurückerhalten.

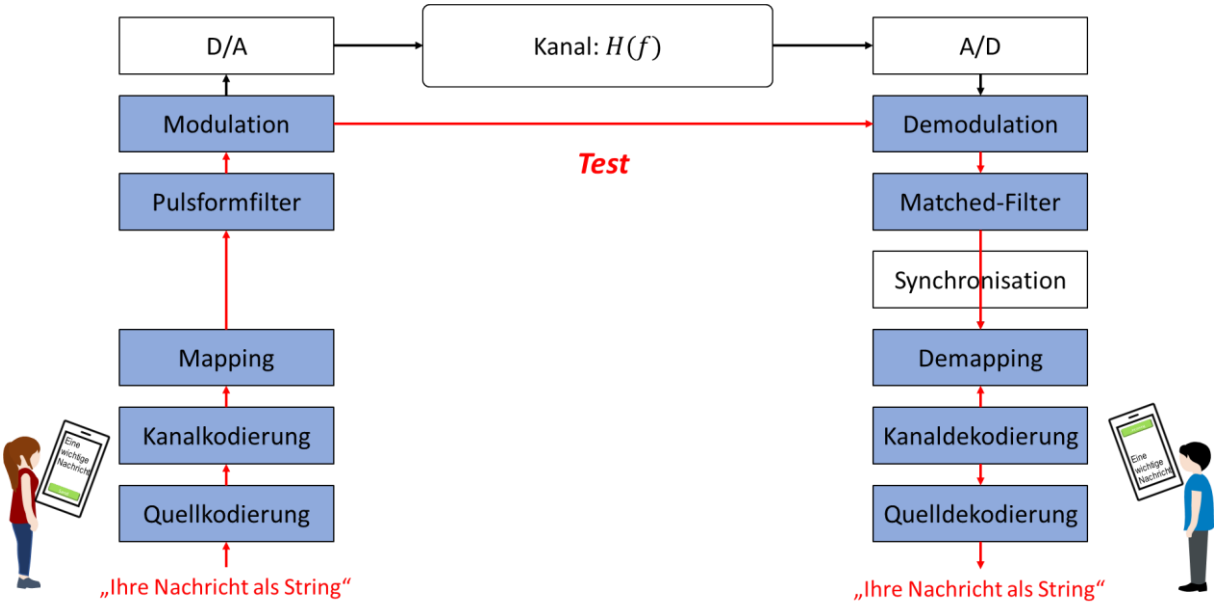
Für einen weiteren Test und eine erste einfache Kanal-Simulation fügen Sie nach dem Sendepulsformfilter Rauschen zu dem Signal hinzu:



Plotten alle bisherigen Diagramme auf der Empfängerseite nun mit den verrauschten Signalen. (Figure 3+4)

Aufgabe 5 – Modulation und Demodulation

Die durch das Pulsformfilter vorbereiteten / geformten Symbole sollen nun auf ein Trägersignal aufmoduliert werden, sodass eine Übertragung über den Kanal möglich wird.



Berechnen und plotten Sie zunächst den Betrag des Spektrums des komplexen Signals nach dem Pulsformfilter (Figure 5).

Jeder Kanal hat (aus technischen und/oder regulatorischen Gründen) ein spezifisches Frequenzband, das genutzt werden kann. Mit der Soundkarte ist prinzipiell der Frequenzbereich $-\frac{f_{sa}}{2} \dots + \frac{f_{sa}}{2}$ nutzbar – allerdings ohne die Gleichspannung. Praktisch kann also $1\ kHz \dots 24kHz$ genutzt werden. Als Trägerfrequenz soll daher $f_C = 10kHz$ verwendet werden.

Legen Sie in Ihrem *„functions“*-Ordner zwei neue Dateien/Funktionen an:

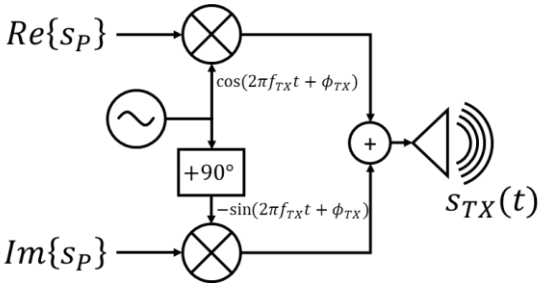
- *„modulation.m“*
- *„demodulation.m“*

5.1 Modulation

Die Modulation bekommt als Eingang:

- die Trägerfrequenz $f_C = 10kHz$
- das komplexe puls-geformte Nachrichtensignal $s_p(t)$
- die Abtastrate des Systems $f_{sa} = 48kHz$

Die Funktion setzt die Modulation wie in folgendem Blockschaltbild um:



Zunächst muss dafür das Lokaloszillatorsignal erzeugt werden – einmal als Cosinus und einmal als -Sinus. Diese beiden Signale haben dieselbe Länge (Anzahl Werte) wie das Eingangssignal der Funktion.

Der Realteil des zusendenden Signals $Re\{s_p(t)\}$ wird dann mit dem Cosinus multipliziert und der Imaginärteil $Im\{s_p(t)\}$ (ohne das ***j***!) mit dem – Sinus. Die Summe der beiden Produkte ergibt das reelle Sendesignal $s_{TX}(t)$. Dieses ist der Rückgabewert der Funktion.

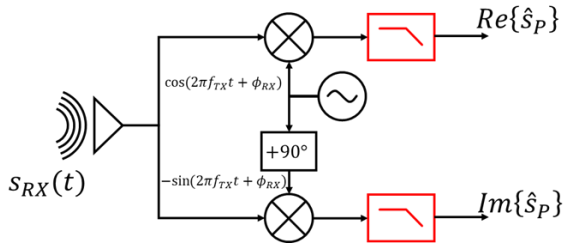
Plotten Sie das Betragsspektrum $|S_{TX}(f)|$ des Sendesignals (in einen zweiten Subplot von Figure 5).

5.2 Demodulation

Die Demodulation bekommt als Eingang:

- die Trägerfrequenz $f_C = 10kHz$
- das reelle Empfangssignal $s_{RX}(t)$ – zum Testen zunächst dasselbe wie das Sendesignal $s_{RX}(t) = s_{TX}(t)$
- die Abtastrate des Systems $f_{sa} = 48kHz$

Die Funktion setzt die Demodulation wie in folgendem Blockschaltbild um:



Allerdings wird hier kein Tiefpassfilter benötigt, da diese Funktion durch das nachgeschaltete Matched-Filter übernommen wird.

Zunächst muss wieder das Lokaloszillatorsignal erzeugt werden – einmal als Cosinus und einmal als -Sinus. Diese beiden Signale haben dieselbe Länge (Anzahl Werte) wie das Eingangssignal der Funktion.

Der Realteil des Basisbandsignals $Re\{\hat{s}_p(t)\}$ wird aus der Multiplikation des Empfangssignals $s_{RX}(t)$ mit dem Cosinus berechnet und der Imaginärteil $Im\{\hat{s}_p(t)\}$ aus der Multiplikation mit dem – Sinus. Das komplexe Basisbandsignals setzt sich dann so zusammen:

$$\hat{s}_p(t) = Re\{\hat{s}_p(t)\} + j \cdot Im\{\hat{s}_p(t)\}$$

Dieses ist der Rückgabewert der Funktion.

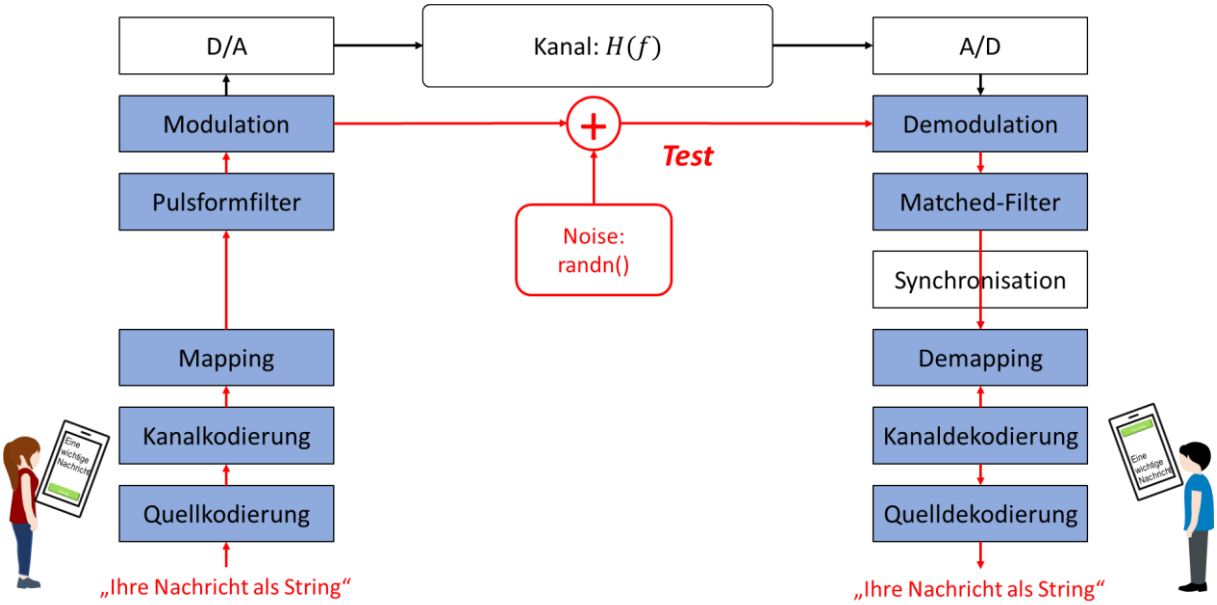
Plotten Sie das Betragsspektrum $|\hat{S}_p(f)|$ des Basisbandsignals (in einen dritten Subplot von Figure 5).

5.3 Zusatz: Kanalsimulation mit Rauschen

Nach der Modulation und vor der Demodulation kann jetzt noch Rauschen für eine einfach Kanalsimulation hinzugefügt werden:

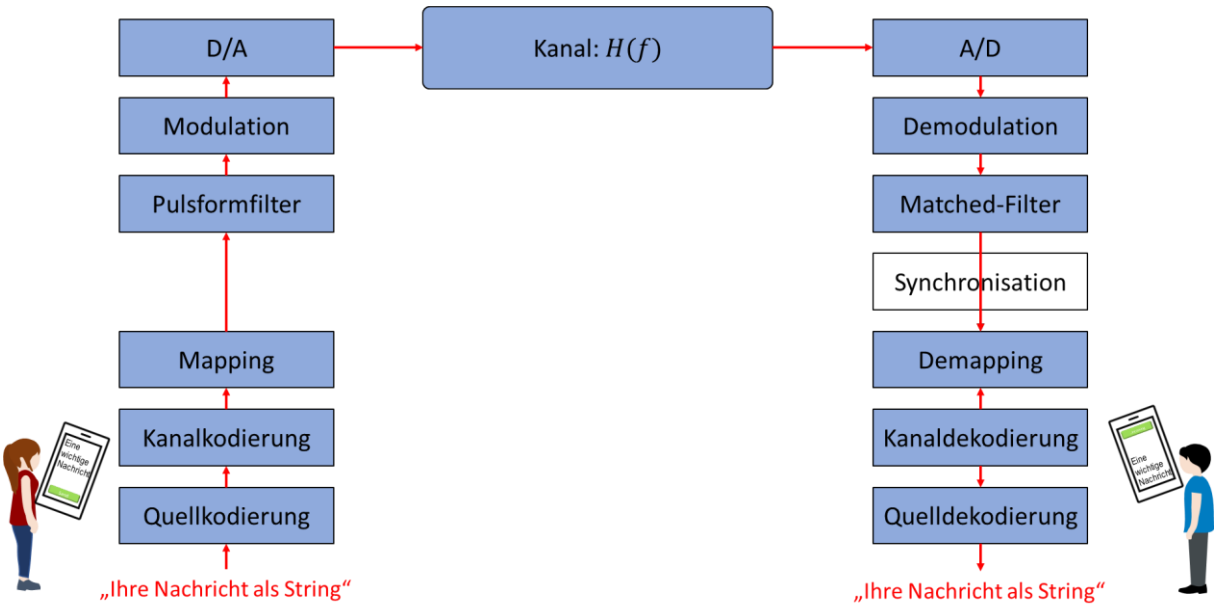
```
%% add noise:
s_RX = s_TX + P_N * randn(size(s_TX));
```

Werte für die Rauschleistung `P_N` können variiert und ausprobiert werden.



Aufgabe 6 – Übertragung über den Kanal

Das Signal ist nun endlich bereit für die Übertragung über den physikalischen Kanal.



Hardware:

Der Kanal ist immer analog. Dazu werden USB-Soundkarten mit einer Abtastrate von $f_{sa} = 48kHz$ verwendet. Schließen Sie zwei Soundkarten an Ihren PC an und verbinden Sie beide von Ausgang zu Eingang mit dem Audiokabel.

In MATLAB:

Fügen Sie zur Initialisierung der Soundkarten am Anfang Ihrer Hauptdatei folgende Zeilen hinzu:

```
%% Prepare audio devices
dac = audioDeviceWriter(fsa,"Device",'Lautsprecher (USB Audio CODEC )');
microphoneID = audiodevinfo(1,'Mikrofon (USB Audio CODEC )');
adc = audiorecorder(fsa,16,1,microphoneID);
```

Die Strings `'Lautsprecher (USB Audio CODEC )'` und `'Mikrofon (USB Audio CODEC )'` identifizieren die angeschlossenen Soundkarten und werden bei Ihrem PC sehr wahrscheinlich abweichen. Wenn Sie den Befehl

devices = `audiodevinfo`

in der Kommandozeile ausführen und sich dann die Variable *„devices“* durch Doppeltklick darauf im Workspace genauer ansehen, finden Sie die genauen Bezeichnungen auf Ihrem PC (unter Input die Audioeingänge und unter Output die Ausgänge).

Wenn die Initialisierung der Soundkarten fehlerfrei durchläuft, kann die Übertragung durchgeführt werden.

Echtzeit-Audiosignalverarbeitung ist in MATLAB nur begrenzt möglich. Um nichts von dem gesendeten Signal am Empfänger zu verpassen, starten Sie als erstes die Aufnahme des Empfangssignals mit dem ADC und warten dann kurz damit die Aufnahme auch sicher gestartet ist:

```
adc.record();
disp('Start')
pause(2)
```

Danach erst starten Sie das aussenden des Sendesignals. Damit hier sicher das komplette Signal auf dem Audioausgang ankommt (Einschalt-/Einschwingvorgänge o.Ä. …) senden Sie davor und danach zusätzliche Nullen:

```
dac.play([zeros(10000,1); s_TX'; zeros(10000,1)]);
```

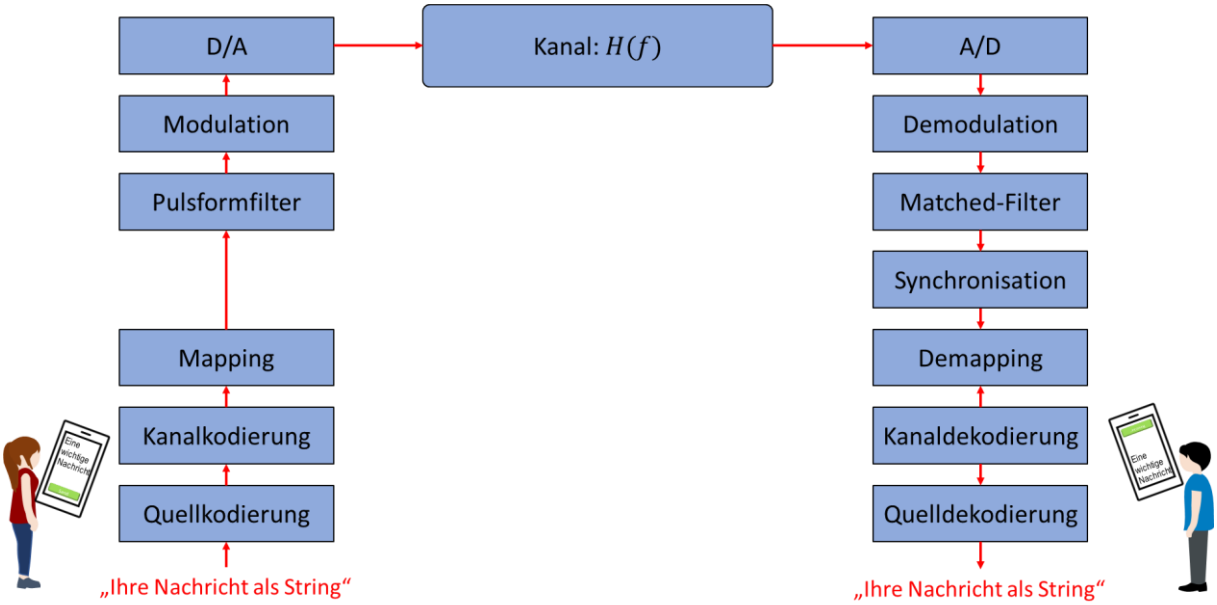
Anschließend muss wieder kurz gewartet werden, bevor die Aufnahme beendet und die Daten abgeholt werden können:

```
pause(2)
disp('Stop')
adc.stop();
s_RX = adc.getaudiodata()';
```

Das Empfangssignal ist – verglichen mit der gesamten Aufnahme – relativ kurz. Vorne und hinten sind sehr lange Bereiche, die nur Rauschen enthalten. Schneiden Sie diese Bereiche manuelle oder automatisiert ab. Dazu ist es hilfreich das Signal einmal einfach über den Index zu plotten.

Aufgabe 7 – Synchronisation

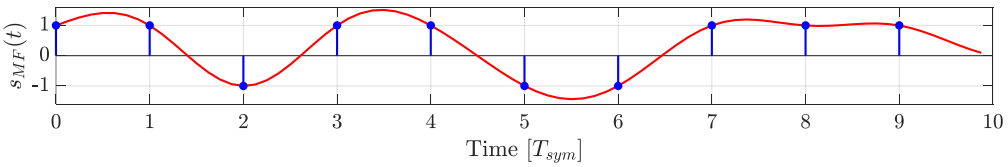
Um dem demodulierten Signal die gesendete Information entnehmen zu können, muss der Empfänger auf den Sender synchronisiert werden.



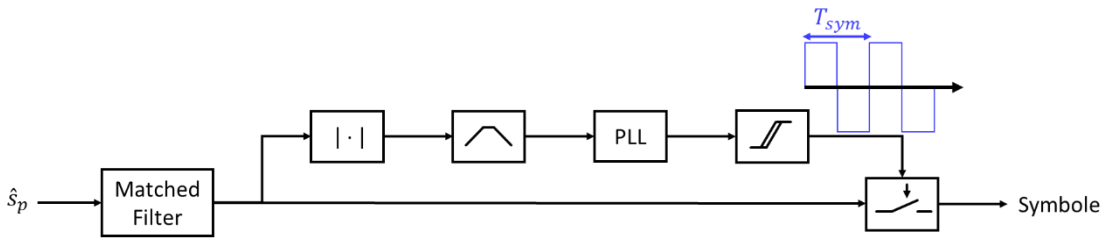
Darunter ist hier zusammengefasst die Symboltaktrückgewinnung und die Kanalentzerrung.

7.1 Symboltaktrückgewinnung

Der Symboltakt gibt an zu welchen Zeitpunkten aus dem Basisbandsignal $s_p(t)$ Amplitudenwerte als Symbolwerte genommen werden müssen, um die gesendeten Symbole zu rekonstruieren.
→ Zeitliche Lage der blauen Symbole auf der roten Kurve in folgendem Bild:



Der Symboltakt ist durch die Symbolübergänge im Betrag (oder auch im Quadrat) des gesendeten und empfangenen Signals enthalten. Zur Symboltaktrückgewinnung wird also zunächst der Betrag des Basisbandsignals $|\hat{s}_p(t)|$ genommen. Anschließend wird dann zur groben Vorselektion ein Bandpass angewendet und schließlich führt eine PLL den Symboltakt nach, um auch Strecken mit mehreren gleichen Symbolen überbrücken zu können. Mit der MATLAB-Funktion sign() kann das Ausgangssignal der PLL noch in ein Rechtecksignal umgewandelt werden, um den Abtastzeitpunkt exakt zu bestimmen:



Die mit dem so rekonstruierten Symboltakt gewonnen Symbole sind durch den Kanaleinfluss noch stark verändert und müssen entzerrt werden.

Um die Kanalverzerrung zu visualisieren, plotten Sie wie schon auf der Senderseite hier nochmal das Konstellationsdiagramm (Realteil der Symbole auf der x-Achse, Imaginärteil der Symbole auf der y-Achse) (Figure 6).

7.2.1 Bandpassfilter

Zur Vorselektion soll ein möglichst schmalbandiges FIR-Bandpassfilter um die Frequenz

$$f_{BP} = \frac{1}{T_{sym}}$$

des Symboltaktes herum verwendet werden. Die Impulsantwort bzw. Filterkoeffizienten eines solchen Filters können so gewählt werden:

$$h(n) = \cos(2\pi f_{BP} n T_{sa})$$

Durch das Design des Pulsformfilters mit $N_{sam} = 8$ Samples pro Symbol, hängen Abtastrate und Symbolrate zusammen $T_{sym} = N_{sam} \cdot T_{sa}$, sodass sich ergibt:

$$h(n) = \cos\left(2\pi n \frac{1}{N_{sam}}\right)$$

Mit $n = 0 \dots N_{BP} - 1$.

Die Länge des Filters bestimmt wie schmal- oder breitbandig das Filter wird. Wählen Sie $N_{BP} = 16$.

7.2.2 PLL

Die Phasenregelschleife kann hier schon grob auf die bekannte Frequenz abgestimmt werden, sodass nur noch Abweichungen davon nachgeregelt werden müssen. Daher wird – in Abweichung zur Übung in der Vorlesung – nicht der Phasenfehler aufakkumuliert und als Argument an eine Cosinusfunktion übergeben, sondern ein gemittelter Phasenfehler als Phase an einen Cosinus mit der bekannten Frequenz übergeben. Der Code der PLL könnte so aussehen:

```
function [output] = pll(input, Nsam, alpha, k)

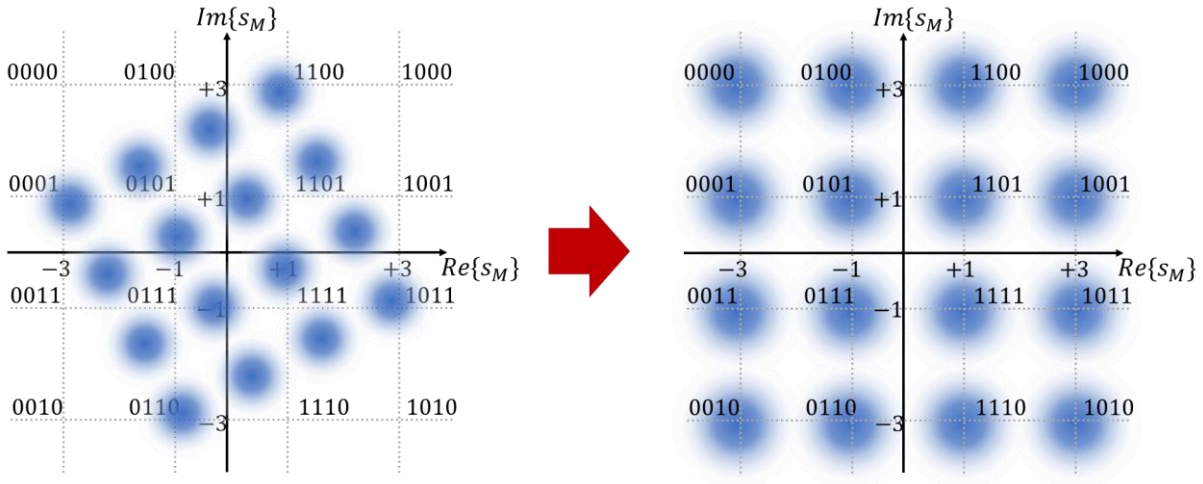
N = length(input);
output = zeros(N, 1);
phaseError = 0;
phaseError_mean = 0;

for n=1:N
    output(n) = cos(2*pi*n/Nsam + phaseError * k);
    phaseError = output(n) * input(n);
    phaseError_mean = alpha * phaseError_mean + (1-alpha) * phaseError;
end
```

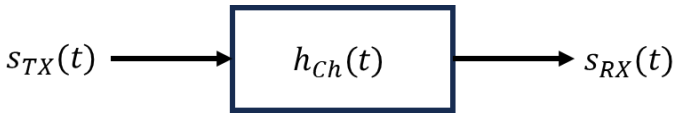
Wählen Sie die Parameter $k = 10$ und $\alpha = 0.99$.

7.2 Kanalentzerrung

Aufgabe der Kanalentzerrung ist es den Kanaleinfluss rückgängig zu machen.



Dazu ist es notwendig den Kanaleinfluss zu kennen bzw. diesen zu ermitteln. Der Kanal wird als LTI-System modelliert:



Um den Einfluss des Systems zu kennen, ist die Kenntnis der Impulsantwort bzw. der Übertragungsfunktion nötig.

Um die Impulsantwort des Kanals schätzen zu können, muss ein in Sender und Empfänger bekanntes Signal gesendet werden. Hierzu wird ein Barker-Code verwendet, da dieser gleichzeitig die Möglichkeit bietet den Start der Nachricht (oder eines Frames) zu identifizieren.

Stellen Sie Ihren Symbolen (vor dem Pulsformfilter) in Real- und Imaginärteil den folgenden Barkercode – multipliziert mit der maximalen Symbolamplitude – voran:

barkerCode = [1 1 1 1 1 -1 -1 1 1 -1 1 -1 1];

Bzw.:

barkerCode =3 * [1 1 1 1 1 -1 -1 1 1 -1 1 -1 1];

Auf Empfängerseite finden Sie den Start dieser Sequenz, indem Sie das gesamte Empfangssignal nach der Symbolabtastung mit derselben Sequenz korrelieren.

Der Einfachheit halber wird angenommen, dass der Kanaleinfluss über das genutzte Frequenzband konstant ist. In dem Fall kann die Übertragungsfunktion schon mit je einem komplexen Sendesymbol und dem entsprechenden Empfangssymbol geschätzt werden.

Beispiel:

Gesendet wurde: $1 + 1j$

Empfangen wurde: $0{,}20 + 0{,}67j$

Dann kann das Empfangssignal entzerrt werden, indem die komplexen Empfangssymbole mit dem komplexen Faktor multipliziert werden:

$$Faktor = \frac{1 + 1j}{0{,}20 + 0{,}67j}$$

Mit dem Barker-Code der Länge 13 kann dieser Entzerrungsfaktor aus 13 Werten (Real- und Imaginärteil separat) gemittelt werden.

Wenden Sie die Entzerrung an und plotten Sie das Konstellationsdiagramm erneut (als zweiten Subplot in Figure 6).